DeFuzz：通过跨机制安全不变量发现编译器静默失效防御的智能体系统

中文阅读版 | 根据 `docs/paper/main.tex` 翻译 | 2026-08-28

摘要

编译器插入的防御机制——栈金丝雀、`_FORTIFY_SOURCE`、CET/IBT、CFI、BTI、PAC 和影子栈——是在内存安全漏洞被触发后必须守住的最后一道防线。它们的有效性不仅取决于防御代码本身是否正确，也取决于对底层指令集架构（ISA）的假设。当某一假设在特定后端上失效时，防御可能发生“静默失效”：程序编译时没有告警，运行时功能仍然正确，但其安全契约已不再得到执行。CVE-2023-4039 即为一例：在 AArch64 上使用变长数组时，GCC 的栈保护器会被绕过；它在其他架构上的同类问题多年未被上游修复。

检测这类静默失效有两个相互耦合的难点。其搜索空间是“防御机制 × ISA”的二维矩阵，每个单元格都由独立的中端 pass 或后端模板支撑。更难的是失效判定：二进制既不会崩溃，也不会与其他编译器产生功能差异，因此基于崩溃或差分预言机的模糊测试无从报告。我们将这种缺乏判定能力的问题称为“预言机缺口”。

本文提出 DeFuzz，一个用于弥合预言机缺口并直接搜索该矩阵的智能体系统。第一，我们将每种防御必须满足的安全不变量——从编译器源码注释、ABI 规范和已确认历史漏洞中提炼而来——系统化地转化为可机器检查的预言机。每个检查器返回四态判定；每项漏洞主张都由确定性的二进制或运行时信号支撑，而非模型输出。第二，我们提出跨机制不变量生成流水线：把一种机制中已确认的根因作为探针，在另一种机制中检索结构相似的代码，并经过类比过滤和蕴含验证来实例化新不变量。基于 GCC 16.1 中 25 个已确认防御漏洞和 24 个探针，该流程产生 11 条新的跨机制不变量，且均不重复已有种子。第三，我们构建显式编排的智能体循环：确定性流水线按固定顺序执行构建、覆盖率与预言机，只在固定位置调用智能体；智能体通过版本化黑板通信，而不是自由对话。与自由运行的智能体系统相比，这一设计提供可复现轨迹、按边消融能力，以及能回溯到确定性证据的漏洞主张；检查器绑定的 ISA 路由还将“机制 × ISA”笛卡尔积压缩为一次语义决策。

> 在 GCC 与 LLVM 上，DeFuzz 发现了 [待补：N] 个静默失效缺陷，其中 [待补：M] 个已获上游确认。

目录

1. 引言
2. 背景与动机
3. 系统概览
4. 编译器防御的安全不变量
5. 跨机制不变量生成
6. 预言机：从不变量到判定
7. 智能体循环
8. 实现
9. 评估
10. 讨论与局限性
11. 相关工作
12. 结论

1. 引言

过去十年，公开披露的内存安全漏洞持续增长。主流厂商与安全研究者日益认识到：仅依靠人工审计和安全编码规范，无法根除复杂上层软件中的内存安全问题。因此，漏洞被触发之后程序的行为变得至关重要。栈金丝雀、`_FORTIFY_SOURCE`、控制流完整性（CFI）、指针认证（PAC）和影子栈等由编译器与运行时共同部署的机制，本应在攻击触发时充当失效保护的最后屏障。

这些防御具有共同的工作模式：编译器分析抽象语法树（AST），插入检查指令、布局约束或元数据，以便在运行时偏离预期时终止程序。栈金丝雀是典型例子：它在局部缓冲区和保存的返回地址之间放置随机哨兵值，并在函数返回前进行验证。

1.1 静默失效

防御机制失效时常常是静默的。CVE-2023-4039 表明，GCC 的 `-fstack-protector` 在 AArch64 上可能被绕过：当函数使用变长数组（VLA）时，后端把金丝雀放在 VLA 上方，保存的返回地址却位于其下方。溢出因而可以改写返回地址，而不触及金丝雀。

这类问题具有“双重静默”特征：编译期工具链不告警，运行时程序也能正确执行预期功能；但安全契约已经完全失效，攻击者可绕过表面上已启用的保护。

1.2 两个耦合挑战

第一，搜索空间很大。编译器必须在“防御机制（如 Canary、IBT、PAC）× ISA（如 x86-64、AArch64、RISC-V）”的二维矩阵中维持安全契约。每个单元格对应一个独立的中端 pass 或目标相关 lowering 模板；相似的遗漏常会在整个矩阵中长期存在。

第二，存在预言机缺口。静默失效不会使二进制崩溃，也不会导致其输出与其他编译器不同。Csmith、YARPGen 等传统测试与模糊测试依赖崩溃或输出差分，因此无法发现只破坏安全契约的逻辑缺陷。

1.3 方法概览

DeFuzz 通过三部分工作弥合这一缺口：把防御应满足的安全不变量转化为可机器检查的预言机；以已确认历史漏洞为探针，在不同机制间检索并泛化根因；以显式编排的智能体循环执行生成、路由、构建、覆盖率和预言机判定。编排器而非智能体决定控制流，智能体仅在固定位置生成种子并通过版本化黑板给出语义化反馈。

1.4 贡献

- 构建确定性、可机器检查的静默失效预言机，所有结论均基于可复现的静态或执行证据。
- 提出跨机制不变量生成流程，将历史漏洞的抽象根因迁移到“机制 × ISA”矩阵中的其他位置，生成 [待补：N] 条新不变量。
- 设计显式编排的智能体模糊测试架构：固定流水线位置、版本化黑板通信，以及检查器元数据驱动的 ISA 路由，保证可复现、可消融、可审计。
- 在多架构 GCC 与 LLVM 上发现 [待补：N] 个此前未知的静默失效缺陷，获得 [待补：M] 个 CVE 与上游补丁。

2. 背景与动机

2.1 后端 lowering 流程中的编译器防御

编译器发出的防御会在编译期加入额外检查指令（如金丝雀比较、CFI 目标检查）、布局约束（如保护值的位置、影子栈槽位）或元数据（如 BTI 落点、PAC 签名），以便在运行时偏离预期时被检测到。这些防御并非在 AST 层一次性插入的单体结构；它们要经历编译器内部表示的多轮 lowering。在 GCC 和 LLVM 中，安全不变量通常在中端 IR（如 GIMPLE 或 LLVM IR）阶段引入，但必须穿过到后端表示（如 RTL、SelectionDAG 或 Machine IR）的转换。

这一过程充满架构相关复杂性。寄存器分配、指令调度和栈帧最终布局受目标 ISA 的 ABI 支配。因此，在目标无关 IR 中看似正确的防御，可能在后端 lowering 时被静默破坏。以 `alloca` 或 VLA 为例，动态栈分配需要目标相关的帧指针调整，这可能意外覆盖或绕过已建立的安全边界。由此，“机制 × ISA”矩阵中的每个格子确实是独立的：一种防御在某个 ISA 上实现正确，并不意味着在另一 ISA 的后端发射模板中仍然正确。

2.2 代表性案例：静默失效

回到 CVE-2023-4039：栈保护器依赖如下不变量——“金丝雀必须位于所有可由溢出触达的栈对象与保存的返回地址之间”。在 AArch64 上，若函数使用 VLA，后端栈帧构造逻辑会把金丝雀放在动态分配区域上方，而保存的返回地址位于其下方。来自 VLA 的溢出会先覆盖返回地址，金丝雀尚未来得及被检查。该问题自早期 GCC 版本就存在，直到 2023 年才被分析并修复。

这同样是“双重静默”问题：编译期没有错误，功能测试也通过，但安全契约已被破坏。类似遗漏还会遗留在其他后端。例如，CVE-2023-4039 的同类问题 PR-96191 在 2020 年修复时覆盖了主流架构，却遗漏了回退后端，使同样的失效在其他目标上又静默存在多年。

2.3 为何现有方法会漏检

崩溃与差分模糊测试只观察表面信号：是否崩溃、是否与另一编译器的输出不同。静默失效按定义保持原程序语义，除非主动利用，否则通常不崩溃，因此这些方法不会报告问题。静态分析又受限于预定义模式，而静默失效在不同后端的表现形式差异很大，难以泛化。自由运行的智能体模糊器则牺牲可复现性：一旦 LLM 自行决定控制流和漏洞判定，执行轨迹难以重放，消融难以隔离，结论也难以追溯到确定性证据，容易产生由幻觉导致的高误报率。

2.4 威胁模型与范围

假设目标应用上层源码中存在潜在、可利用的内存安全漏洞，如缓冲区溢出；攻击者能够触发该漏洞，并试图劫持控制流或执行任意代码。本文仅关注编译器自动插入的安全机制（如栈金丝雀、`_FORTIFY_SOURCE`、CFI、IBT）本身的正确性与有效性：若防御机制在干净编译后未能按设计检测或缓解攻击，则属于研究范围。我们只描述被违反的不变量，不构造可工作的利用。硬件漏洞（如推测执行/Spectre）、侧信道、业务逻辑问题，以及通过内联汇编等方式显式绕过防御的源级错误，不在范围内。为保持证据与复现条件稳定，RAG 语料和实验评估均固定在 GCC 16.1。

3. 系统概览

DeFuzz 将离线知识构建与在线漏洞搜索分离。离线阶段读取编译器源码、规范与历史漏洞记录。一次分段的 CoT 审阅提供广泛的候选提取；基于历史漏洞的 RAG 则集中搜索可能共享已知根因的实现。通过证据落地和去重的候选项会进入不变量目录，并被实现为静态或动态检查器。已确认的新发现还可以加入历史漏洞池，作为新的检索探针。

在线阶段通过编排对生成种子的编译与后续执行，检查编译器对防御的发射结果。运行初始化时，编排器将检查器目录展开为固定队列。每个目标对应一对“不变量—检查器”；检查器元数据给出适用的 ISA。该目标在其轮次或墙钟时间预算耗尽前保持不变。

每轮中，生成器产生一个 C 种子，随后固定流水线执行：路由 → 构建 → 覆盖率 → 预言机。Pass、NotApplicable 和 Error 进入反馈分支后开始下一轮；其中 Error 作为基础设施诊断保持可见，消耗当前轮次，但不被视为漏洞。Fail 则触发最小化器，并用同一检查器复检。所有中间状态和结果均版本化保存，以支持回放与审计。

4. 编译器防御的安全不变量

4.1 什么是安全不变量

安全不变量是某种防御要想有效就必须满足的性质。违反它意味着：无论源码中是否存在显式缺陷，该机制在编译产物或运行时已被静默削弱。每条不变量都记录一个可机器验证的“陈述（statement）”，并单独记录“观测（observation）”字段；后者只描述违反时可从外部观察到的现象，而不规定检测手段。

这种分离是有意的。例如，“二进制中缺少 `__stack_chk_fail` 符号”是一个现象；“在 ELF 的 `.dynsym` 节中查找该符号”则是实现预言机时采用的检测手段。

4.2 调研方法与来源

不变量档案来自三类来源：（1）编译器源码注释和断言；（2）编译器与 ABI 文档；（3）已确认历史漏洞及其补丁。每条记录均采用统一模式：ID、statement、compiler、version、target、source_kind、evidence、version_sensitivity 和 observation。这样，下游消费者——如预言机、静态分析或 CI——可独立决定如何使用每条不变量。

语料规模超过单一模型上下文，相关证据也分散在遥远的源码文件、规范和补丁中。因此，本文按防御机制、编译器阶段和函数边界切分材料。一次分段 CoT 提示要求模型在每个片段中识别受保护资产、激活条件、所需安全性质及外部可观测的违反现象。该步骤只产生候选项而非已接纳的不变量；所有候选项必须保留源证据，并通过后文的证据落地检查。

4.3 自底向上的分类法

我们不预先设定自顶向下的类别，而是以数据为先，对已收录的 468 条不变量进行聚类，再从聚类中归纳根因族，以避免预设偏差。跨机制、跨 ISA 反复出现三类问题：

- 退出时敏感寄存器/状态残留：快速或异常退出跳过了防御所依赖的寄存器或状态清零。
- Stack-clash 栈帧大小截断：强制整数窄化将很大的帧大小翻成负数，导致探测循环被跳过。
- RISC-V 大地址物化：在拆分或合成大地址/偏移时，意外的符号扩展使计算地址发生偏移。

4.4 可机器检查的形式：静态与动态

每条不变量按违反信号是否可在未执行的二进制中观测来分类。符号、节、反汇编和字符串属于静态信号；退出状态、输出标志和寄存器快照等则严格依赖执行轨迹。分类考虑可观测性、判定性、

计算成本和归因能力。若一个不变量同时具有静态与动态违反信号，系统会将其拆分成不同检查器，而不是合并成单一启发式规则，以保持精度。

5. 跨机制不变量生成

5.1 动机：抽象失效模式迁移

分段审阅覆盖面广，但当编码同一根因的证据分散在不同机制名称、编译器阶段或目标后端时，它往往无法把这些孤立片段聚合起来。检索增强生成（RAG）提供了一条互补且高产的路径：把已确认历史漏洞作为明确探针，主动搜索结构相关的实现。它不保证完备性，却能显著提高发现共享已知根因的同类缺陷的概率。

RAG 首先将一个已确认漏洞提炼为与机制无关的失效签名，随后在已索引的中端和后端语料中检索语义结构相近的实现。本文刻意不采用以实体为中心的检索，例如匹配 API 或符号名，因为这会把结果限制在原始防御机制内部。查询围绕抽象操作、被违反的假设和受保护资产构造，使一个探针能够发现不同机制和后端中的类似漏洞。

5.2 流水线：提炼、类比、专门化与蕴含

泛化流程分为四步。第一步，将已确认根因提炼为抽象描述。以 CVE-2023-4039 为例，原始描述“AArch64 上 VLA 覆盖金丝雀”可提炼为抽象语义签名：“动态大小的局部对象绕过了静态计算出的安全边界”。第二步，类比过滤器判断检索到的代码块是否体现同一结构性失效，主动剔除表面词汇相似但根因不同的结果。第三步，在具体目标源码上下文中对不变量进行专门化，例如将抽象边界映射为影子栈的具体栈帧计算。第四步，通过蕴含验证确认形成的陈述能够从引文证据中逻辑推出。分段 CoT 审阅和 RAG 路径最终汇入同一个证据落地与新颖性过滤器。

5.3 BM25 与向量嵌入互补

为有效查询代码库，每个提炼出的根因都会形成双视角查询：`root_cause_phrase` 为稠密向量提供抽象语义摘要；`critical_tokens` 则捕获寄存器名、宏开关等机制相关标识符，供稀疏检索使用。

在同样的 24 个探针种子和 4,496 个 GCC 16.1 源码片段上，本文比较了稀疏 BM25 与稠密 `doubao-embedding-vision` 检索。BM25 擅长匹配 `critical_tokens`，例如 `ix86_zero_call_used_regs`、`SUBREG_PROMOTED` 和 `CONST_HIGH_PART`；稠密检索则根据 `root_cause_phrase` 找到词法锚点较弱但语义同构的代码块，例如 CMSE 清零集合构造器和 RISC-V 饱和截断逻辑。

按 `(seed_id, chunk_id)` 和陈述去重后，两条路径有 5 条交集、4 条仅由 BM25 找到、2 条仅由嵌入检索找到，共形成 11 条并集结果；经阈值为 85.0 的新颖性检查后，这 11 条均被标记为新颖。

5.4 证据落地门与可复现性

分段 CoT 审阅和历史漏洞 RAG 产生的候选项进入同一接纳路径。只有在两道静态证据门确认引文能够支撑其陈述和观测，且新颖性检查确认其不重复已有种子或人工规则后，不变量才被接纳。由于稠密检索器每次调用可能返回略有差异的向量，系统按查询文本缓存查询向量，以保证 top-k 排序及最终接纳集合可跨运行复现。新的发现（包括零日）一旦复现并确认根因，就会按相同模式序列化并加入探针池，使下一轮搜索能继续查找该根因的其他实例。

6. 预言机：从不变量到判定

6.1 检查器设计

为了将已接纳的不变量转化为可执行预言机，本文定义形式化的检查器契约。令 P 表示已编译程序空间，I 表示目标 ISA 集合，S 表示已接纳不变量集合，则预言机是如下确定性函数：

$$O(P, I, S) \rightarrow \{\text{Pass}，\text{Fail}，\text{NA}，\text{Error}\}$$

即：对一个编译后的程序、目标 ISA 和安全不变量，返回通过、失败、不适用或错误。无需执行即可判定的不变量实现为静态检查器，分析符号、节或反汇编；需要运行时上下文的性质，例如退出状态和内存状态，则实现为动态检查器。对既可静态又可动态判定的不变量，系统会冗余实现两类检查器，以确定性精度优先于启发式近似。

每个检查器都返回四态结果。机制关闭或种子未触发该机制时返回 NotApplicable，聚合层会忽略它。只有确定性检查器失败，或执行差分能够复现时，发现才被计为已确认；绝不以模型输出本身作为确认依据。

若程序化检查器无法判定某个“不变量—ISA”单元格，则返回 NotApplicable 或 Error。LLM 被严格禁止根据反汇编或符号证据推断、覆盖判定；所有 Fail 都必须来自确定性的执行或静态分析规则。因此，任何暂定聚合结果都不会仅因模型输出被提升为“违反”，从而保证最小化过程和最终漏洞主张都植根于可复现证据。

6.2 机制聚合器与开关配置

按机制划分的聚合器合并各检查器结果：任一 Fail 都会产生携带证据的机制级违反。防御开/关的参数配置允许预言机进行差分判定；由于检查器依据二进制和运行时信号，而不是产生该二进制的编译器，GCC 与 LLVM 可以共享同一套检查器。

6.3 检查器元数据：唯一事实源

每个检查器声明三类元数据：它绑定哪些 ISA、它是单 ISA 还是差分检查器、以及其成本是低还是高。这些元数据同时被 gRPC 与 MCP 适配器读取，是唯一事实源；因此第 7 节中的 ISA 路由只是一次表查找，而不是智能体决策。

7. 智能体循环

7.1 设计原则

目标选择与控制流由编排器而非智能体掌握。运行初始化时，编排器构造固定的“不变量—检查器”目标队列。对于每个目标，检查器元数据决定适用 ISA；目标在配置的轮次或时间预算内保持不变。每一轮严格按相同顺序执行：生成 → 路由 → 构建 → 覆盖率 → 预言机 → 分流。智能体只在固定位置被调用，通过共享状态通信。生成器负责提出候选；预言机栈负责裁决；确认发现必须具备确定性证据。

7.2 显式编排与黑板

智能体构成闭环：反馈引导下一轮生成，预言机裁决结果，未违反情况更新反馈参数，违反则触发最小化。关键在于，所有智能体间通信均经由版本化黑板，而非直接对话。

算法可概括为：对目标队列中的每个目标，在预算未耗尽时，生成器根据黑板反馈生成种子；系统针对该目标 ISA 编译种子；预言机返回判定 v 与覆盖率 cov。若 v 为 Fail，则最小化器在同一预言机下缩减种子并产出 PoC；若 v 为 Pass 或 NA，则反馈智能体根据 cov 和判定写回下一轮指导；每轮结束后预算减一。

该架构带来三项性质：固定黑板版本即可确定性固定智能体输入，因此可复现；连接边可独立开关，因此可做精确消融；每项漏洞主张均能追溯到确定性执行证据，因此可审计。

7.3 三类智能体

生成器在轮次起点调用，拥有只读的源码搜索和不变量查询工具，为分配的“不变量—检查器”对输出种子。覆盖率不是由智能体报告，而由覆盖率节点测量并回馈，因此智能体无法伪造覆盖率。

反馈智能体在未违反时调用，是上下文隔离的子智能体：它将覆盖率增量、检查器身份及 Pass/NotApplicable 判定转化为下一轮建议，并写回黑板。

最小化器在违反时调用。它以确定性的 delta debugging（C-Reduce）为主力，LLM 只提供语义指导，以免缩减时删去触发漏洞的关键结构。每个候选缩减均须重新编译，并由同一检查器复检，才会保留。

7.4 检查器路由的种子与 ISA 绑定检查器

确定性队列中的每个目标对应一条不变量及其检查器。路由节点根据元数据，将检查器扩展到所有适用 ISA；生成器从不自行选择目标不变量或 ISA。差分检查器始终展开到其完整 ISA 集合，避免跨 ISA 信号被模型行为剪枝。低成本静态检查器作为旁路保持启用，但不改变生成器的指定目标。

当检查器返回 Pass 或 NotApplicable 时，反馈智能体记录相关上下文，同一目标对继续运行直到预算耗尽。Error 走同样的非漏洞路径，作为可见基础设施诊断并消耗本轮。Fail 则将种子和确定性证据交给最小化器。编排器推进前，黑板会记录两条分支的状态。

8. 实现

DeFuzz 将体系结构拆分为高性能评估核心和基于 Python 的智能体编排器。用 Go 实现的核心服务提供确定性的构建执行、覆盖率追踪和预言机裁决。它一方面通过 gRPC 向编排器提供高吞吐的程序化访问，另一方面通过模型上下文协议（MCP）向智能体提供只读工具，用于源码搜索、不变量查询和调用 delta debugging 工具 C-Reduce。

预言机定义、静态/动态检查器和 ISA 适用规则均集中在 Go 核心中，构成所有元数据的唯一事实源。编排器负责显式智能体循环，严格维持生成、路由、构建和预言机裁决的拓扑顺序。它管理版本化黑板，记录种子语料的谱系、累积覆盖率指标和反馈建议。为支持严格审计，每一次执行都会持久化一个独立的工件包，其中包含序列化状态链、完整工具链配置与 LLM 超参数。该设计支持对任何已确认漏洞主张进行确定性重放和事后归因。

9. 评估

本节目前仍待实验数据补全。

实验设置需要明确 LLM 参数（模型、温度等）、基线配置（自由运行智能体的设置），以及用于可复现的准确计算预算和硬件环境。

RQ1 | 真实漏洞：需要补充已确认漏洞数量和 CVE 编号。

RQ2 | 不变量生成质量：需要由专家抽样评级，并计算 Cohen's κ。评级类别包括：真实安全性质、合理但非必要、平凡、错误。

RQ3 | 消融：需要评估显式编排相对于自由运行基线、覆盖率反馈和预言机证据落地各自对命中率与成本的贡献。

RQ4 | 跨 ISA 泛化：等待实验数据。

10. 讨论与局限性

本节目前仍待实验数据补全，最终应至少涵盖：

1. 将不变量转换为检查器所需的人工工作量分析；
2. LLM API 的 token 成本与预算分析；
3. 误报分析，尤其是 QEMU 模拟保真度与裸机硬件之间差异造成的影响。

11. 相关工作

编译器缺陷发现与预言机缺口

传统编译器模糊测试在发现优化缺陷和崩溃方面非常成功。Csmith、YARPGen 和 EMI 等工具以差分测试或崩溃检测作为预言机。但它们对安全防御存在结构性盲区：静默失效按定义保持源码的功能语义，编译产物能正确执行，输出也与未优化或其他编译器构建的结果一致。既无语义分歧也无运行时崩溃，因此差分与崩溃预言机在理论上无法检测这类安全契约违反。

编译器中的静默与安全缺陷

编译器静默安全失效正受到更多关注。“Silent Bugs Matter”等研究给出了由编译器引入的安全缺陷的经验分类，说明优化可能意外删除安全检查，或引入时序侧信道。然而，这些研究主要是测量工作，尚未提供可自动、生成式地跨不同后端发现新实例的系统。DeFuzz 将这些观察操作化为可执行预言机，从而填补这一缺口。

基于 LLM 与 RAG 的性质生成

近期工作开始用 LLM 推导系统性质。PropertyGPT 使用 RAG 合成智能合约不变量，SpecAuditor 则通过语义泛化识别规范违反。DeFuzz 的关键区别在于检索策略：现有方法多以实体为中心，如匹配特定 API 或变量名；DeFuzz 则有意拒绝实体驱动匹配，采用抽象失效模式迁移，并用双视角查询跨越不同防御机制（如栈金丝雀与 PAC）和架构间的词汇鸿沟。

LLM 智能体模糊测试：幻觉与确定性

将 LLM 作为自主智能体接入模糊测试流水线，例如 AgentFuzz，能带来强大的定向探索能力。但自由运行的智能体架构存在严重的可复现性与幻觉问题：如果 LLM 可以自行判断复杂输出是否是“漏洞”，系统就不可避免地产生由模型幻觉驱动的误报。DeFuzz 通过显式编排在结构上区别于此类方案：LLM 被限制在生成和反馈角色，所有漏洞主张都必须由程序化检查器的确定性执行结果支撑。因此，幻觉被排除在裁决阶段之外，每个报告的违反均是可复现、可操作的缺陷。

12. 结论

编译器插入的安全防御是抵御内存安全攻击的重要后备屏障，但在后端 lowering 过程中，这些防御的有效性经常会因静默失效而受损。这类问题既能绕过传统的崩溃与差分模糊测试预言机，也会让二进制看似受保护、实则在“防御机制 × ISA”矩阵中仍存在功能性漏洞。

DeFuzz 面向这一根本盲区。通过将防御语义提炼为确定性、可机器检查的预言机，系统把静默失效的发现从临时性人工审计转化为系统化、可复现的过程。跨机制不变量生成流水线表明，历史根因能够被抽象并迁移，用于发现不同防御机制中的类似问题。显式编排的智能体循环进一步说明：只要将智能体输出严格锚定在确定性执行证据上，LLM 就能有效服务于复杂模糊测试任务，而不会在裁决环节陷入幻觉。

最终，DeFuzz 在生产级编译器中发现了 [待补：N] 个此前未知的静默失效，并形成 [待补：M] 个 CVE。这些发现既揭示了当前编译器防御的脆弱性，也说明在编译器开发生命周期中集成严格、以不变量为基础的预言机是必要的。

说明：本文档是原英文论文的中文阅读译本。参考文献、图表及原始图示未在本阅读版中复刻；原稿中的实验占位符已保留。